Group Coursework (40%)

Title: Document Search and Information Retrieval System in C

Summary

Project theme: Design and implement a document search engine in standard C that builds an index over a collection of text documents and returns ranked search results.

Programming restriction: The system must be implemented in ANSI C or a newer standard using standard C libraries. No machine learning models, LLMs, embedding models, semantic-search APIs, or ready-made retrieval frameworks may be used.

Group size: THREE (3) to FIVE (5) students per group (across lab groups 1 and 2).

Assessment weightage: 40% of the course.

Submission deadline: 14 December 2026 (Monday) before 23:59 (tentative)

1.0 Background & Motivation

Search engines allow users to locate useful information from a large collection of documents without reading every document manually. Although modern commercial and AI-powered search systems may use sophisticated semantic models, the underlying information retrieval problem can also be studied using *fundamental algorithms and data structures*.

A document search engine normally performs several stages: document loading, text preprocessing, indexing, query processing, matching, ranking, and presentation of results. One important structure is an inverted index, which maps a word or term to the documents in which that term occurs. This is conceptually similar to the index at the back of a book.

In this coursework, students will build a **Document Search and Information Retrieval System in C**. The system will process a collection of English text documents, create an index, accept user search queries, retrieve matching documents, calculate a relevance score, and display the ranked results.

The project is deliberately designed without AI or machine-learning components. Students are expected to solve the information retrieval problem using C programming, algorithms and data structures such as *arrays, structures, pointers, dynamic memory, linked lists, file processing, searching and sorting*.

Example

Suppose the collection contains the following documents:

```
doc01.txt: Renewable energy can reduce dependence on fossil fuels. Solar
power is widely used.
```

```
doc02.txt: Wind energy and solar energy are important renewable energy
sources.
doc03.txt: Public transport can reduce traffic congestion in cities.
```

For the query "renewable energy", a simple search engine may:

- normalize the query into the terms renewable and energy;
- look up these terms in the inverted index;
- identify `doc01.txt` and `doc02.txt` as matching documents;
- calculate a relevance score based on the number/frequency of matched terms;
- sort the matching documents by score; and
- display the ranked results together with useful information such as document ID, title/file name and score.

The objective is not to reproduce a commercial search engine. The objective is to demonstrate a working information retrieval system while applying the C programming and algorithmic techniques taught in COMP1028.

2.0 Project Overview

This is **group project of THREE (3) to FIVE (5) members** per group (Note: students can form group across Monday and Tuesday lab groups).

Each group is responsible for designing, implementing, testing and documenting one complete Document Search and Information Retrieval System in standard C.

Your program must achieve the following core functions (but is not limited to them):

1. Load and manage a collection of text documents.
2. Read and preprocess document text.
3. Tokenize text into searchable words/terms and normalize them.
4. Build an inverted index mapping terms to documents and/or term frequencies.
5. Accept one or more user search queries.
6. Retrieve documents containing the query terms.
7. Calculate a deterministic relevance score.
8. Sort and display the top matching documents.
9. Provide document and index statistics.
10. Save useful system output, such as search results and statistics, to files.
11. Handle invalid input, missing files, empty documents and other reasonable edge cases.

The implementation must demonstrate meaningful use of the C programming concepts taught in COMP1028. A submission that merely prints pre-defined answers or performs a linear scan of the entire

document collection for every query without implementing an index will not satisfy the intended project objective.

Tentative Recommended System Pipeline

```
Document files → File Loader → Tokenizer/Normalizer → Inverted Index → Query Processor
                                                ↓
                                  Matching → Scoring → Sorting
                                                ↓
                                      Ranked Results / Report
```

3.0 Objectives

At the end of this project, students are expected to demonstrate at least the following capabilities:

1. Apply *string manipulation, arrays, structures, pointers and dynamic memory* in C.
2. Perform *file processing* involving multiple input and output files.
3. Design and implement *functions* and modular programs.
4. Design an appropriate *data structure* for an inverted index.
5. Apply *searching, sorting, counting* and *frequency-analysis* algorithms.
6. Develop a deterministic *document retrieval and ranking method*.
7. Analyse algorithmic choices using time, memory and implementation considerations.
8. Develop a *menu-driven interface* with *error handling*.
9. Design and conduct *systematic functional and performance tests* and *benchmarking*.
10. Explain and *present* the implemented algorithms, data structures, C concepts and engineering decisions clearly.

4.0 Key Functionalities and Features

The requirements below define the minimum expected system. Students may choose their own implementation approach and internal data structures, provided that the resulting system is functional, understandable and demonstrably written in C.

1. Document Input and File Processing

- Load multiple **.txt** documents from a specified folder or configured input collection.
- Process a collection containing at least 50 documents and at least 100,000 words in total (unless an alternative corpus is specified by the module convenor).
- Preferably English only (can be multi-language which is an optional advanced feature).
- Read documents using standard C file-processing functions.
- Assign or maintain a unique document ID for each document.
- Handle missing, empty or unreadable files without crashing.
- Store sufficient metadata to identify a retrieved document.

2. Tokenization and Text Normalization

- Convert words to a consistent case, normally lowercase.
- Handle punctuation and common non-letter separators consistently.
- Split document text into searchable terms.
- Ignore empty tokens.
- Count total terms and unique terms for each document or the collection.
- Use a stopword list or another clearly documented deterministic rule if stopword removal is implemented.

3. Inverted Index

- Build an *inverted index* from the document collection.
- Map each unique term to the document(s) in which it occurs.
- Store at least the document ID and enough information to support relevance scoring.
- Support lookup of terms without reading every original document for every query.
- Use structures and pointers appropriately; a linked-list-based posting list is recommended though other feasible data structures may be considered.
- Release dynamically allocated memory appropriately.

4. Search and Query Processing

- Accept a query from the user.
- Normalize the query using the same basic rules as the documents.
- Support single-term and multi-term keyword queries.
- Retrieve documents that match at least one query term.
- Clearly indicate when no documents match.
- Display a configurable number of top results, e.g. top 5 or top 10.

5. Relevance Ranking

- Assign a deterministic relevance score to each matching document.
- The minimum scoring method should consider query-term occurrence/frequency in the document.
- Sort matching documents by score in descending order.
- Handle ties consistently, for example using document ID or file name.
- Document the scoring formula in the report.

6. Statistics and Reporting

- Display collection statistics such as number of documents and total/unique terms.
- Display useful per-document statistics.
- Display index statistics such as number of unique terms and posting relationships.
- Save search results and/or statistics to `report.txt`, `report.csv`, or another clearly documented output file.

7. Menu-Driven User Interface

- Provide a clear command-line interface with console menu (or optional GUI as advanced feature).
- Suggested options (but not limited to them):
  - Load Collection
  - Build Index
  - Show Statistics
  - Search
  - View Document
  - Save Results/Report
  - Exit
- Provide clear instructions, prompts and error messages.
- Prevent invalid menu choices or malformed input from causing crashes.

8. Error Handling and Robustness

- Handle missing or invalid input files.
- Handle empty documents and empty queries.
- Handle queries containing punctuation or repeated spaces.
- Handle queries with no matching terms.
- Avoid obvious memory leaks and invalid pointer use.
- Avoid crashes caused by ordinary invalid user input.
- etc.

9. Persistent Index (Recommended Core/Good Feature)

- Provide an option to save the constructed index or relevant index data to a binary/text file.

- Provide an option to reload the index without rebuilding it from all documents, where feasible.
- Document the file format used by the program.
- Use standard C file functions such as `fopen, fread / fwrite/ fprintf/ fscanf/ fseek/ ftell/ rewind` where appropriate.

10. (Optional) Suggested Advanced Features for Distinction

- Graphical User Interface (GUI)
- TF-IDF or another well-justified deterministic relevance-scoring method.
- Phrase search, e.g. "renewable energy".
- Boolean AND / OR queries.
- Wildcard search, e.g. comput*.
- Search-result snippets showing where query terms occur.
- Hash-table-based term indexing and comparison with a linked-list index.
- Binary-search or sorted-array term lookup.
- Index persistence and fast index loading.
- Document preview and term-frequency visual summaries using text output.
- Support for an additional text-based document format without using an external AI/semantic retrieval system.
- Performance comparison with advanced metrics not suggested in this coursework specification, e.g. Mean Average Precision (MAP)@K, NDCG@K, effect of collection size on search time, effect of query length or query selectivity on retrieval, etc.
- Any other features not listed as minimum expected system from items 1-9 above (require module convenor approval).

Note: Advanced features are not required for a basic passing implementation. They should only be attempted after the core search engine is reliable.

4.1 Programming Restrictions

- The core system must be implemented in standard C (ANSI C or C11/C17/C23 or another newer standard).
- Use standard C libraries such as `stdio.h, stdlib.h, string.h, ctype.h, math.h, time.h` etc. where appropriate.
- x Do not use pre-trained or externally trained AI models.
- x Do not use LLM APIs, embedding APIs, semantic-search APIs or cloud AI services.
    - x Transformer embedding models (e.g., BERT, E5, BGE, MiniLM, Nomic Embed)
    - x Large Language Models (LLMs) (e.g. OpenAI GPTs, Claude, Gemini, Deepseek, Llama etc.)
    - x Sentence embedding APIs or cloud AI services (SQLite, Boost, OpenCV, Torch, TensorFlow etc.)
- x Do not use ready-made information-retrieval/search-engine frameworks that perform indexing or

ranking on behalf of the program.

- x Do not submit a program that delegates the core indexing, search or ranking tasks to another language or external executable.
- Third-party libraries may only be used for clearly documented non-core purposes (require module convenor approval).

5.0 Marking Rubric

Criteria		Deliverables & Artefacts	Poor	Good	Excellent	Mark (/100)
1	**Functional Product, Evaluation & Benchmarking**	• Program binary and supporting files • Program source codes • Final Report • Final Software demo Video Recording (Week 12 - 1 per group)	• One or more core functions are missing or do not work reliably. • Program frequently produces incorrect results, crashes, or fails on invalid/edge-case inputs. • Benchmarking is missing or uses insufficient evidence to assess performance. • Results/conclusions are largely unsupported by test or benchmark data. • Poor and difficult user interface • No achievement of any advanced feature beyond required features and functions	• All major core functions are implemented and generally work correctly. • Program handles normal inputs and most prescribed invalid/edge cases correctly. • Benchmarking uses the prescribed dataset and at least the required performance/quality measures. • Results are interpreted with basic evidence-based conclusions. • Good and easy console-based user interface • No achievement of any advanced feature beyond required features and functions	• All required core functions work correctly and reliably. • Program passes all or almost all prescribed tests, including invalid/edge cases, without crashes. • Benchmarking is complete and uses the prescribed dataset and required metrics. • Results include meaningful comparisons and are supported by measured data. • Very good and easy console-based user interface • One or more advanced features (e.g. graphical user interface) are implemented correctly, without compromising core functionality.	**30**
			0 - 10	11 - 20	21 - 30	
2	**Algorithm Design & Implementation**	• Final Report • Final Presentation	• One or more major algorithms are missing, inappropriate, or	• Appropriate algorithms and data structures are used for the main	• Algorithms and data structures are appropriate, efficient,	**30**

Criteria		Deliverables & Artefacts	Poor	Good	Excellent	Mark (/100)
		Video Recording (Week 12 – 1 submission per group)	incorrectly implemented. • Data structures are unsuitable or used incorrectly. • Significant errors occur in C concepts such as pointers, memory management, structures, arrays, strings, or file processing etc. • Algorithm/data-structure choices are not adequately explained. • Poor code quality (with errors and duplications) and lack modularity and documentation	functions. • Core C concepts are implemented correctly with only minor errors or inefficiencies. • Code is reasonably modular (minimal duplications), with functions used to separate major tasks. • Codes are sufficiently documented. • Dynamic memory, pointers and file processing are handled appropriately where required. • Major algorithm/data-structure choices are explained with basic justification.	and correctly implemented. • C concepts are applied correctly, including pointers, dynamic memory, structures, strings and file processing where relevant. • Excellent code quality with modular design (no duplication) and complete documentation • Important algorithm/data-structure choices are justified using relevant trade-offs such as time, memory or implementation complexity. • Code demonstrates strong control of the underlying algorithms rather than merely producing the required output.	
			0 - 10	11 - 20	21 - 30	
3	**Engineering Process & Evidence**	• Git commit history • Engineering Evidence Log	• Little evidence of planned or incremental / continuous development. • Git/engineering records are absent, minimal, or	• Development plan identifies major tasks, responsibilities and milestones. • Git/engineering records	• Development is clearly planned and supported by continuous evidence across the project period.	**20**

Criteria		Deliverables & Artefacts	Poor	Good	Excellent	Mark (/100)
		• AI Usage Log • Final Report • Individual Interim Contribution Video Recording (Week 6 – 1 submission per member)	show mainly final-stage development. • Problems encountered and how they were resolved are rarely recorded. • AI Usage Log is missing, incomplete, or does not show verification of AI-assisted work.	show development across multiple stages. • Engineering records describe significant problems and corresponding solutions. • AI-assisted work is disclosed and basic verification/testing is documented.	• Git/engineering records show meaningful implementation, testing, debugging and iteration rather than only final submission activity. • Comprehensive testing evidence records. • Engineering records show specific problems, solutions and resulting improvements. • AI Usage Log identifies the AI-assisted task/component and documents how the output was tested, verified and modified where necessary. • Evidence is consistent with the submitted software and individual contributions.	
			0 - 8	9 - 14	15 - 20	
4	**Individual Technical Quality**	• Engineering Activity Log • Individual Interim Contribution Video Recording (Week 6	• Cannot clearly identify or demonstrate their own technical contribution. • Cannot adequately explain important parts of their contributed code.	• Can clearly identify and explain their main technical contribution. • Can explain the purpose and operation of their contributed code with	• Clearly and precisely explains their technical contribution and its relationship to the overall system. • Demonstrates detailed	**20**

Criteria		Deliverables & Artefacts	Poor	Good	Excellent	Mark (/100)
		– 1 submission per member)	• Cannot explain the relevant algorithm, data structure or C concepts used. • Cannot correctly trace or predict the behaviour of their code for a given input. • Cannot identify or correct a simple error/change in their contributed code.	minor gaps. • Can explain the relevant algorithm, data structure and C concepts used. • Can trace their code and predict its output for a given test input. • Can reason about their code make/explain a simple modification or debugging change.	understanding of their contributed code, including important design decisions. • Correctly explains relevant algorithms, data structures and C concepts and their trade-offs. • Can independently reason and explain meaningful modification to their contributed code and predict its effect. • Demonstrates understanding that is consistent with the submitted implementation and engineering evidence.	
			0 - 8	9 - 14	15 - 20	
					Total	**100**
					Assessment Weightage	**40%**

Note:

- Criteria 1 – 3: Marks for all members of the group will be the same
- Criteria 4: Marks for each member may not be the same.

6.0 Datasets

The coursework will use a collection of suggested English plain-text documents. The purpose of providing a common corpus suggestion is to make functional testing and performance comparison more consistent across groups.

6.1 Coursework Document Corpus

- Documents will be supplied as **.txt** files and organized into a collection/folder.
- The core development/training corpus will contain a sufficiently large collection for indexing and testing, normally at least 50 documents and 100,000 words in total.
- Suggested dataset documents from public-domain:
- **Stage 1 — Development**
  - 20 Newsgroup subset
    - https://archive.ics.uci.edu/dataset/113/twenty+newsgroups
    - https://www.kaggle.com/datasets/crawford/20-newsgroups
    - https://scikit-learn.org/stable/datasets/real_world.html
    - http://qwone.com/~jason/20Newsgroups/
  - It is a general-purpose corpus for you to practice building your search engine.
  - It contains about 18,846 text documents across 20 topics, with predefined training and testing subsets.
- **Stage 2 — Validation**
  - Cranfield Collection subset:
    - https://huggingface.co/datasets/irds/cranfield
    - https://www.kaggle.com/datasets/hhhoang/cranfield-dataset
    - https://github.com/oussbenk/cranfield-trec-dataset [1]
  - It is an actual classical Information Retrieval collection. The collection contains approximately:
    - 1,400 documents
    - 225 queries
    - 1,837 relevance judgments (qrels) - The qrels specify which documents are relevant to which queries, which means you can evaluate and validate the search engine you build objectively.
- **Stage 3 — Hidden final benchmark**
  - The final benchmark corpus may contain documents that were not used during early development so that the system is evaluated on unseen inputs.
  - The exact benchmark files and release schedule will be announced through Moodle (**not less than 14 days before the submission deadline**).

6.2 Required Use of the Dataset

- Use the development corpus suggested (Stage 1 and 2) while designing and testing your system.
- Your program must work on the suggested data collection rather than treating the supplied examples as

[1] TREC text files use simple, tag-based delimiters (similar to HTML or early SGML) to separate distinct document boundaries and fields.

fixed cases.

- Do not hard-code document names, query answers, rankings or expected benchmark results.
- Run the final benchmark (Stage 3) using the dataset and query set specified by the module convenor.
- Include benchmark results in the final report.

6.3 Suggested Test Query Categories

Category	Example	Purpose
Single term	renewable	Basic index lookup.
Common term	energy	Produces many matches.
Rare term	geothermal	Tests selective retrieval.
Multiple terms	renewable energy	Tests multi-term scoring.
No match	quantumbanana	Tests no-result handling.
Punctuation	renewable-energy	Tests normalization.
Repeated term	energy energy	Tests query normalization.
Phrase / advanced	"renewable energy"	Optional phrase-search feature.

7.0 Sample Use Case Scenario

The following scenario illustrates one possible system workflow. It is an example only; students may design a different workflow.

7.1 Initial Setup

1. User selects: Load Document Collection
2. Program loads 100 text files.
3. Program reports: 100 documents, 128,450 total terms, 18,320 unique terms.
4. User selects: Build Index
5. Program creates an inverted index and reports index statistics.

7.2 Search

```
Search query: renewable energy

Top 5 results:
Rank  File             Score       Matched Terms
1     doc_024.txt      12.50       renewable, energy
2     doc_071.txt       9.00       renewable, energy
3     doc_013.txt       6.50       energy
...
```

The exact scores above are illustrative only. Each group must define and document its own required scoring formula.

7.3 Example Basic Scoring Formula

A simple mandatory scoring approach may be:

```
score(document, query) = sum of query-term frequencies in the document
```

For example, if the normalized query contains renewable and energy, and a document contains renewable 3 times and energy 5 times, its basic score may be 3 + 5 = 8. Students may improve the formula, but the formula must be deterministic, implemented in C, and explained in the report.

7.4 Example Inverted Index Representation

```
Term: renewable
    -> Document 24 (frequency 3) -> Document 71 (frequency 2) -> ...


Term: energy
    -> Document 13 (frequency 4) -> Document 24 (frequency 5) -> Document 71
(frequency 7) -> ...
```

A possible C design is to represent terms and posting lists using structures and linked pointers. Other designs are allowed if they meet the functional requirements and demonstrate appropriate C programming.

8.0 Benchmarking & Performance Evaluation

The project must include measurable evaluation rather than only screenshots showing that the program runs.

8.1 Recommended Comparison

Compare a baseline linear scan with the implemented indexed search.

Experiment	Method	Measure
A	Linear scan of document text for each query	Search time and other performance metrics (see below)
B	Inverted-index search	Search time and other performance metrics (see below)
C (advanced)	Alternative index such as hash table	Search time + memory/implementation trade-off

Students should not claim that one approach is faster merely because it is theoretically expected to be faster. Use measured data and discuss factors such as collection size, number of unique terms, number of postings, lookup method and query selectivity.

8.2 Suggested Performance Metrics

8.2.1 Retrieval-quality metrics

Precision@K

Measures the proportion of relevant items in the top K results

For example:

Precision@5 means of the first 5 returned documents, how many are relevant?

$$Precision@K = \frac{\text{relevant documents in top K}}{K}$$

Recall@K

Measures whether the search engine actually finds the relevant documents.

For example:
Recall@10 or Recall@20 depending on the number of relevant documents per query.

$$Recall@K = \frac{\text{relevant documents retrieved in top K}}{\text{total relevant documents}}$$

Mean Reciprocal Rank (MRR)

Measures "How quickly does the search engine find a relevant document?" (user experience)

For example:
Suppose the first relevant document occurs at:
Rank 1 → 1/1 = 1.00
Rank 2 → 1/2 = 0.50
Rank 5 → 1/5 = 0.20
Rank 10 → 1/10 = 0.10
Then:

$$MRR = \frac{1}{Q}\sum_{q=1}^{Q}\frac{1}{rank_q}$$

8.2.2 Suggested Performance Metrics

- Index construction time:
  - Time to load + tokenize + construct index
- Query latency
  - average search time per query
- Throughput
  - queries/second
- Number of indexed terms
- Number of posting entries
- Worst-case query latency
  - Useful when some queries will retrieve thousands of postings
- Memory usage
- etc.

9.0 Deliverables and Submission Instruction

The deliverables for submission and assessment consist of the following key components:

1. **Software Binary (.exe) and Source Codes (.c)** & configuration files (+ any other related artefacts for code documentation and to make the program run) (Week 12)

2. **Git commit history log** (Week 1 – 12)
   - Provide Git repository (Github link) in Final Report
3. **Reports** (Week 12)
   - Engineering Evidence Log – refer to COMP1028_Engineering_Evidence_Log_Template.docx
   - AI Usage log – refer to COMP1028_AI_Usage_Log_Template.docx on Moodle
   - Final Report – refer to COMP1028_Project_Final_Report_Template.docx
4. **Video Recordings**:
   - **Individual Contribution Presentation**: Each member records a short contribution video demonstrating their technical work and explaining their relevant C concepts, algorithms and data structures. (1 submission per member to Moodle): **3 – 5 min** (Week 6)
   - **Final Group Presentation/Demonstration/Answers to Q&A** (1 submission per group to Moodle by Team Leader): **7 – 12 min** (Week 12)

     Suggested Final Video Structure

Part	Minimum guide	Content
Presentation	≥ 3 min	Project overview, architecture, objectives and key engineering decisions etc.
Software Demo	≥ 4 min	Load documents, build/load index, perform searches, show ranking/statistics and benchmark evidence etc.
Viva/Q&A	≥ 3 min	Answer module convenor-provided questions demonstrating technical understanding.

Note:
- Viva questions for Q&A in the video recording will be released on Moodle.
- Upload all individual or group video recordings to cloud (e.g. OneDrive, Youtube) and provide URL in Final Report

All files, documents and links to github repository and video recordings are to be submitted on **Moodle**.
Deadline: **14 December 2026 (Monday) before 23:59** (tentative)

End – All the best!